Benchmarking gRPC and RESTful Services for Real-Time IoT System

ANNA NORDIO, University of Padua, ITA

In the design of high-performance distributed systems, especially within IoT telemetry, optimizing communication efficiency and minimizing protocol overhead are known and challenging problems. This work examines the performance trade-offs between the REST architectural style and the gRPC framework. A benchmarking study is conducted under equivalent conditions to evaluate both approaches, showcasing how gRPC minimizes latency and network overhead compared to traditional JSON-based RESTful services. This is particularly true in high-frequency data exchange, however REST remains more widely used thanks to its simplicity and interoperability. The evaluation is also extended to consider interactions in web-based contexts, highlighting the implications of both approaches in environments where browser compatibility and data transport efficiency are relevant. The focus of this work is to highlight the trade-offs between architectural flexibility and raw performance in distributed monitoring environments.

CCS Concepts: • **Distributed Systems**; • **gRPC**; • **RESTful Services**; • **Simluated IoT Systems**;

Additional Key Words and Phrases: gRPC, REST, Protocol Buffers, HTTP/2, Telemetry Benchmarking, Envoy Proxy, API Performance

ACM Reference Format:
Anna Nordio. 2026. Benchmarking gRPC and RESTful Services for Real-Time IoT System. 1, 1 (April 2026), 11 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In the context of modern distributed architectures the ability of individual nodes to exchange data efficiently is a critical aspect. As the number of connected devices increases, a typical challenge in the Internet of Things (IoT) domain, the choice of communication protocol becomes an important factor for overall performance and responsiveness. This is particularly relevant in systems where latency and scalability directly affect data consistency and system reliability. The REpresentational State Transfer (REST) architectural style has long been widely adopted due to its simplicity and broad usage. However, its reliance on JSON and HTTP overhead can become a limitation in scenarios where frequent or continuous data transmission is required, such as telemetry collection. In such scenarios, even small protocol inefficiencies can lead to significant performance degradation at scale.

As an alternative, gRPC uses Protocol Buffers and HTTP/2 allowing a more compact message format and more efficient use of network resources and improving performances in real time systems. This makes gRPC particularly suitable for applications requiring low-latency and high-throughput communication. Modern distributed systems are no longer limited to server-to-server communication, as they increasingly operate in different environments that also include web-based components. This further emphasizes the importance of understanding how communication paradigms behave across different deployment settings.

Author's Contact Information: Anna Nordio, anna.nordio@studenti.unipd.it, University of Padua, Padua, ITA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

2 Architectural Styles vs. Invocation Mechanisms: REST and gRPC

To better interpret the results of this study, it is necessary to introduce the concepts of REpresentational State Transfer (REST) and gRPC. Although they represent an architectural style and an invocation mechanism respectively, both are commonly used for data exchange and communications in distributed systems. The following subsections describe their main characteristics and design principles.

2.1 The REST Architectural Paradigm

REST (Representational State Transfer) is a resource-oriented architectural style built on the HTTP protocol. Each resource is identified by a URI and manipulated via standard methods (GET, POST, PUT, DELETE). Its stateless nature ensures that each request contains all necessary processing information, enhancing scalability at the cost of potential overhead due to redundant metadata in HTTP headers. In modern implementations, REST typically relies on JSON for data exchange. While JSON is favored for its human-readability and broad interoperability, it is a text-based format that incurs higher serialization costs and larger payload sizes compared to binary alternatives. Despite these overheads, REST remains the standard for public APIs and web-based integration layers due to its simplicity and the ubiquitous nature of HTTP/1.1. [2]

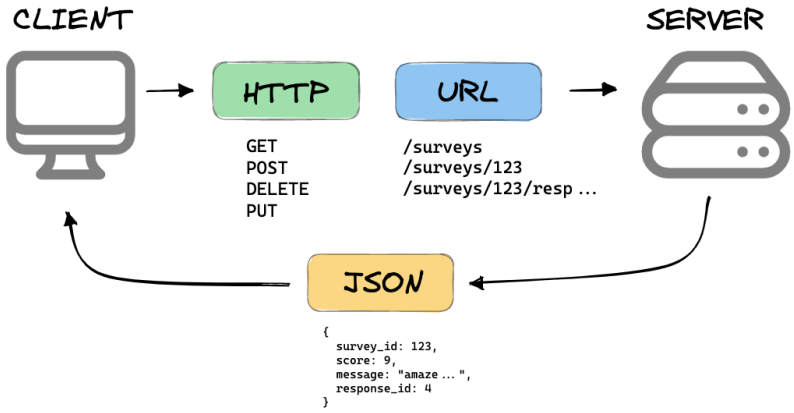


Fig. 1. API REST model. [Public domain], via mannhowie.com (https://mannhowie.com/rest-api).

2.2 The gRPC Communication Framework

The Remote Procedure Call (RPC) paradigm, established in the late 1970s, was designed to simplify distributed computing by allowing programs to execute functions on remote servers as if they were local. gRPC modernizes this concept by replacing traditional, often cumbersome RPC implementations with a streamlined, contract-first approach optimized for high-density microservices. gRPC is a high-performance RPC framework that evolves the traditional Remote Procedure Call model for cloud-native architectures. It relies on Protocol Buffers (protobuf) as its Interface Definition Language (IDL) and binary serialization format. Unlike JSON, protobuf produces compact binary payloads and enables efficient code generation of client and server stubs, significantly reducing CPU overhead during serialization. The framework

operates over HTTP/2, leveraging features such as binary framing and multiplexing to handle multiple concurrent streams over a single TCP connection. This architecture eliminates the head-of-line blocking typical of HTTP/1.1 and reduces connection negotiation latency. Supporting unary, server, client, and bidirectional streaming, gRPC is optimized for microservices where strong typing, contract-based interfaces, and low-latency communication are critical. [1].

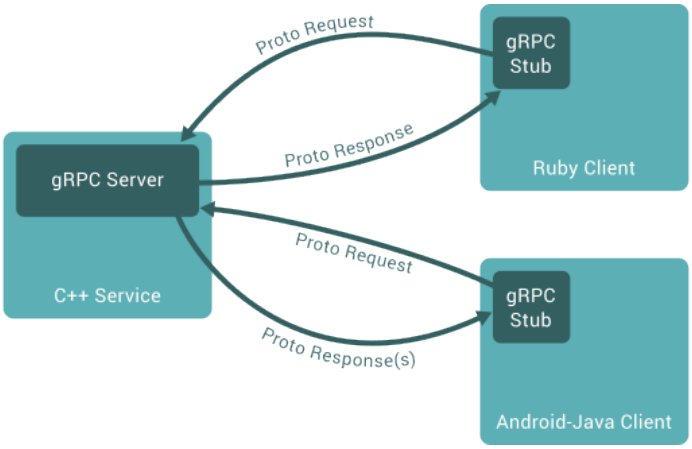


Fig. 2. gRPC communication model. [Public domain], via grpc.io (https://grpc.io/img/landing-2.svg).

3 Technologies and Architectural Decisions

This section describes the system architecture and the technological choices made for the implementation of the benchmarking system. The primary objective is to provide a controlled environment capable of simulated high-frequency telemetric traffic to evaluate the performances of the different communication methods. In the following sections, the choice of Go as core language for the project will be analyzed, followed by a description of the internal components of the system designed to support both RESt and gRPC communication, from the generation of data to the display of real-time data on a web page. Finally, we will describe how the benchmarking suite was implemented to gather the data necessary for the final evaluation.

3.1 Choice of Go for Implementation

The choice of Go as the language for the implementation of the PoC was made to ensure the accuracy and scalability of the benchmark, while minimizing the "observer effect" where measurement overhead alters final results. Thanks to the concurrency model based on goroutines, the system is able to coordinate hundreds of independent virtual sensors, avoiding the bottlenecks typical of context-switching in traditional threads. Furthermore, Go is natively integrated with Protobuf and HTTP/2, allowing us to directly manipulate network metadata and implement lock-free state management logic through atomic operations. The adoption of an asyncronous pipeline to gather data for the different metrics ensures a complete separation of telemetry transmission from the statistical analysis phase, removing the possibility that computational overhead and memory I/O could interfere with the precision of temporal measurements, making the comparison between REST and gRPC very rigorous and representative of a real high-level performance scenario.

3.2 System Overview

The proposed system is designed to evaluate the performance trade-off of REST and gRPC in a high-throughput IoT context. The architecture is structured as a decoupled layered platform to guarantee the computational and monitoring operations do not interfere with ingestion operations (the "Observer effect").

Virtual Sensor Layer. This layer simulates through independent goroutines an arbitrary number of sensors sending a payload of a determined size. The quantity of sensor and the size of the payload is defined at runtime by the user or by the benchmark suite. Every sensor works with a sampling frequency of 10 Hz (100 ms), managed through the `time.Ticker` primitive. To overcome the limits of TCP Head-of-Line Blocking, a connection pool of 100 gRPC clients is implemented. Each new sensor is assigned to a new client using a Round-Robin strategy defined in the function `syncSensors`, ensuring a more uniform load distribution. The sensor lifecycle is managed through atomic synchronization mechanisms (`atomic.Value`), allowing for instantaneous reconfigurations without the bottlenecks typical of mutex locks. In table 1 it is possible to observe the specific composition of the different payloads.

Payload	Dominant Fields	Technical Structure & Rationale
Small	Scalars	Basic telemetry (ID, Temp, Hum, TS) + 2 static tags. Establishes the baseline for minimal signaling.
Medium	String List	Base fields + **200 dynamic strings** (e.g., *"tag_category_name_X"*). Challenges string allocation and escaping logic.
Large	Float Map	Base fields + **2000 Key-Value pairs** (String-Float32). Simulates high-density historical buffers, forcing heavy numerical parsing.
Nested	Object Tree	Base fields + **100 complex objects** with nested metadata maps. Evaluates the computational cost of recursive tree traversal.

Table 1. Structural composition of benchmarking payloads.

Gateway Layer. The gateway layer is the orchestrator and dual-stack server that exposes two dedicated endpoints for the data traffic:

- *gRPC Stream Handler*: Where the native HTTP/2 stream is handled;
- *REST Handler*: Where standard syncronous calls are elaborated on HTTP/1.

To preserve the precision of measurements during stress-tests, the gateway isolates the calculation of critical metrics (such as latency, marshalling times and throughput) in a buffered asyncronous channel (`metricsChan`). This creates a non-blocking separation between the ingestion logic and the analysis layer as a dedicated MetricWorker consumes the channel aggregating results and persisisting them on a CSV log. This specific design ensures that the I/O latency doesn't backpropagate to the network handlers.

Observability Layer. While the core benchmark operates on a direct native path, the system integrates an observability layer for real-time monitoring through a React dashboard. The frontend uses the hook `useTelemetry` to retrieve data (both HTTP and gRPC-Web via Envoy Proxy) to show a comparative analysis between the two methods. The dashboard sets the configuration of the number of sensors, payload size and the active protocol, propagating control signals to the

gateway in real-time. During the benchmark sessions this layer get bypassed so that the metrics gatherd during the session are not altered by the browser polling and the overhead introduced by the translation of gRPC-Web.

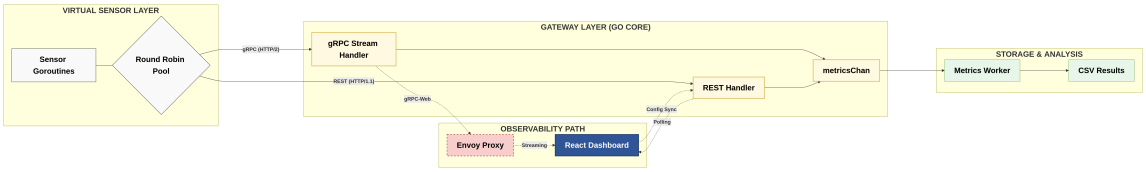


Fig. 3. System architecture

3.3 Gateway Initialization: Dual-Stack Server Architecture

The gateway architecture is designed to work as a dual-stack server, ensuring that both communication methods are simultaneously available to receive traffic without any activation latency. The initialization process occurs in the `main` function of the gateway, using the native concurrency model of Go.

- **gRPC Server**: The server is instantiated on a dedicated port (:50051), registered via `pb.RegisterTelemetryServiceServer` and launched in a dedicated goroutine. This allows the server to handle thousands of streams at one time using the Go runtime's light threads;
- **HTTP (REST) Server**: At the same time, a `http.ServeMux` is configured to handle the REST endpoint as `/telemetry` and `/results`. The HTTP server is distributed on port :8080 using `http.ListenAndServe`. The HTTP server also exposes the control API used by the dashboard to set the runtime configuration. The integration of CORS middleware (`enableCORS`) allows the React frontend to communicate with the gateway even when served from different domains.

These separated servers are coordinated, ensuring that the tests stay on isolated network ports while sharing the same Metrics Worker, making the gathering of data centralized and coherent.

3.4 Telemetry Protocols: REST vs. gRPC

In order to exhaustively analyze the performances, the system implements four distinct communication patterns. This granularity is necessary to isolate the serialization efficency (JSON vs Protobuf) from the transport efficency (HTTP/1.1 vs HTTP/2) and from the invocation model (Unary vs Streaming).

Unary models (Polling). In this configuration, defined in the code as `conf.Mode == "polling"`, every data packet is considered an independent transaction. It is managed by the function `executePolling`.

- **REST Unary**: Handled by the `sendRest` function. Each data point is serialized into JSON and transmitted via a POST request. Despite using a persistent pool connection (`MaxIdleConns: 500`), every call has the entire overhead of HTTP/1.1 headers;
- **gRPC Unary**: Executed with the `grpcClient.SendData` function. It follows the request-response model, but the binary transport on HTTPS/2 and the 100-client connection pool initialized in the `main` drastically reduce connection negotiation times compared to REST.

Streaming models (Native vs. Simulated). This configuration, activated when `conf.Mode == "streaming"`, aims to maximize the throughput by reducing setup costs for every single message.

- **gRPC Native Streaming**: Implemented through the `executeStreaming` function that uses the interface `stream.Send(data)`. Following the initial opening of the stream (`grpcClient.StreamData`), the channel remains persistent. This enables the transmission of binary frames with minimal overhead (approximately 14 bytes per message as calculated at the gateway level) eliminating the repetition of the headers;
- **REST High-Frequency Polling (Simulated Streaming)**: HTTP/1.1 does not support natively client-server streaming, the system emulates this behavior with high-frquency polling. In the `executeStreaming` function, the call to the `sendRest` function happens synchronously. This is a fundamental choice for the integrity of the benchmark as it provides a fair comparison to gRPC. Although Go facilitates background processing through the use of asynchronous goroutines, this approach would have falsified the results. By forcing a syncronous call, the system and the sensor experience network pressure as it has to wait for the completion of HTTP/1.1 RTT cycle before sending the next message. This choice prevents the Go runtime to mask the inefficiencies of REST in this specific case through its scheduling capabilities, making sure that the latency measured is the actual overhead of the transport layer rather than the host's concurrency performance.

3.5 Real-time Observability and Frontend Integration

To enable immediate visual comparison and dynamic test management, the system includes an analytical dashboard developed in React. This component not only serves as a data visualizer, but also acts as a control center orchestrated through various bridging technologies.

Communication Architecture and Proxying). The frontend communicates with the backend (Gateway) through two distinct channels, reflecting the dual nature of the benchmark:

- **REST/WebSocket (Port 8080)**: It is used to get the aggregated results `/results`, the configuration of test parameters (size of payload, number of sensors, etc.) and the monitoring of the gateway health through a WebSocket (`useConnection`);
- **gRPC-Web via Envoy Proxy (Port 8081)**: Browsers do not natively support pure gRPC as it is based on HTTP/2. In order to test the native stream of gRPC from a web interface, Envoy Proxy was introduced. This proxy works as a translator, receiving gRPC-Web calls from the frontend and forwarding them to the gRPC server on port :50051.

Bundling e Protocol Buffer Management. A critical point in the integration is the management of the files generated by the Protobuf compilator for Javascript. The adopted solution is based on the bundling via Webpack. The generated files (`telemetry_pb.js`) utilize CommonJS, which is not directly compatible with modern build tools such as Vite (the one used in this case) or with the standard ES Modules of browsers. Using the `webpack.config.js` configuration, the proto files are packaged into a UMD bundle. This "isolation" resolves the internal dependencies `google-protobuf` and `grpc-web`, exposing a clear interface(`telemetry-proto-bundle`) that the frontend imports as a normal library.

Data Ingestion Mechanisms: Polling vs. Streaming. The dashboard implements two drastically different data reception logics, designed to highlight the performance differences between the two communication methods.

- **Polling Logic (Pull-based)**: Through the hook `useTelemetry`, the client actively interrogates the gateway every 500 ms (`setInterval`). This mode reflects a "snapshot" model, where the update rate is limited by the client timer and each request incurs the entire HTTP negotiation overhead.
- **Streaming Logic (Push-based)**: It is used for the native gRPC streaming. In this case, the client establishes a single persistent connection (`getGrpcStream`). The gateway is the one that pushes data towards the dashboard as soon as they are available. This is possible via the asynchronous event `stream.on('data')`, guaranteeing a better visual fluidity and a minimal update latency, as there is no repetitive polling.

3.6 Docker Orchestration

The infrastructure is designed to be modular and reproducible through Docker, separating the dependencies from the execution. The file `Dockerfile.proto` is dedicated to dependency provisioning, including the various plugiins and guaranteeing a deterministic generation of the code. In this way, any discrepancies between local environments and conflicts of version are eliminated. A second `Dockerfile` uses multi-stage build to compile the Go binaries and the React Bundle (via Webpack). Orchestration is done using separate Docker Compose files to isolate test scenarios. The first stack is dedicated to the dashboard and real-time observability, while a second specific stack is tailored for benchmarking, optimized to minimize background processes and computational noise. This differentiation is enforced through environment variables that allow the system to dynamically configure its behavior. Specifically, these variables are used to deactivate the dashboard and the associated telemetry logic during high-intensity tests. This ensures that the system inhibits CSV write operations for non-essential real-time monitoring data, thereby preventing dataset contamination and reserving disk I/O resources exclusively for the persistence of benchmark results. All services are interconnected via a virtual bridge network (`telemetry-net`) that isolates the test traffic from external interference.

3.7 Benchmarking Setup

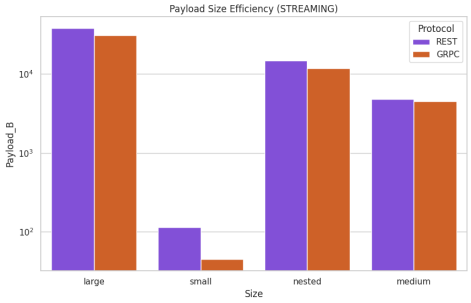
The automated benchmarking system is divided into three phases: execution of the suite, persistence of data and post-elaboration analysis. The execution is managed by a runner that tests 64 unique combinations (4 number of sensors × 2 modes × 4 paylaod sizes × 2 protocols). The system applies internal shuffling to test groups and alternates between load levels to eliminate any bias. Each test is composed of a 30 second warm-up, a 180 second steady-state and a 30 second cool-down to release resources. In steady-state, the `MetricsWriter` saves the data into a CSV file, creating a new file for every single combination (`rotateFile`). The last phase of the benchmarking consists of a Python script that compares REST and gRPC on equal terms and returns some graphs on the metrics saved in the CSV files.

4 Evaluations

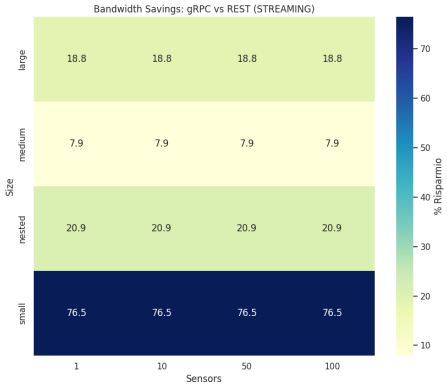
This section analyzes the results obtained from the benchmark suite, comparing the performance of REST and gRPC in terms of latency, payload efficiency and scalability. For brevity, the following paragraphs focus on the analysis of streaming mode cases, which represent the maximum achievable performance; however, a complete and detailed collection of all the graphs produced (including polling mode) can be found in "Appendice Tecnica" (the document is in italian).

4.1 Payload Efficiency and Bandwidth Savings

The first indicator of gRPC's efficiency is the drastic reduction in the size of transmitted data. While REST relies on the verbosity of the JSON format, gRPC leverages the binary serialization of Protocol Buffers. As shown in Fig. 4a, gRPC



(a) Payload Size Comparison



(b) Bandwidth Savings Heatmap (%)

Fig. 4. Analysis of data transfer efficiency: absolute sizes and percentage savings.

reduces the space occupied by data, especially in small messages, where the absence of typical JSON text keys allows for critical savings. This efficiency is most evident in Fig. 4b,in tests with small messages, the bandwidth savings exceed 76%, remaining above 20% even with complex or nested data structures, confirming the effectiveness of binary encoding regardless of sensor load.

4.2 Latency and Scalability under Load

The latency was evaluated by measuring the total response time as the concurrency (number of simultaneously connected sensors) increased. The scalability graph reveals a clear trend in the performance of the two protocols. As the number of sensors increases, gRPC consistently maintains lower average latency than its REST counterpart, regardless of the payload size used. The most significant aspect that emerges from the data is the outperformance of REST by theoretically more expensive gRPC configurations. In particular, we observe that the latency measured for gRPC with a medium payload is consistently lower than the latency of REST with a small payload. Using HTTP/2 multiplexing, gRPC manages hundreds of flows over a single TCP connection. This avoids the overhead of creating new connections and managing large textual headers, allowing gRPC to maintain high performance even with 100 active sensors.

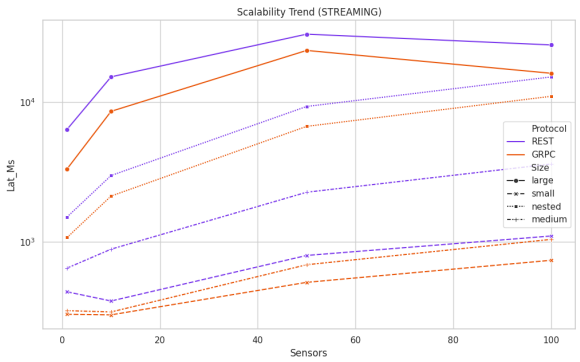


Fig. 5. Scalability Trend: gRPC vs REST

4.3 Latency Breakdown: Marshalling and CPU Impact

To isolate the causes of performance advantage, we analyzed the latency breakdown between network logic and CPU processing times.

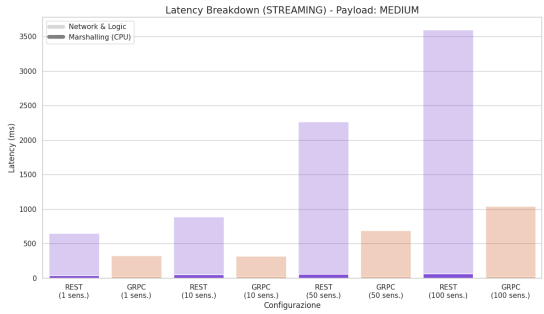


Fig. 6. Latency Breakdown (Medium Payload)

The latency breakdown for medium-sized payloads (Fig. 6) shows that the share ascribable to marshalling is almost negligible in gRPC. In contrast, in REST, the computational cost of parsing and generating JSON strings becomes a visible bottleneck as message complexity increases. The network component in gRPC benefits not only from the smaller binary payload size but also from the compression of HPACK headers provided by HTTP/2. This translates into shorter transit times, solidifying gRPC as the most efficient choice for high-throughput telemetry systems where minimizing CPU overhead is critical.

4.4 Marshalling Gain

A thorough analysis of computational efficiency reveals that the advantage of gRPC is not limited to network traffic reduction alone, but lies significantly in data processing speed (marshalling).

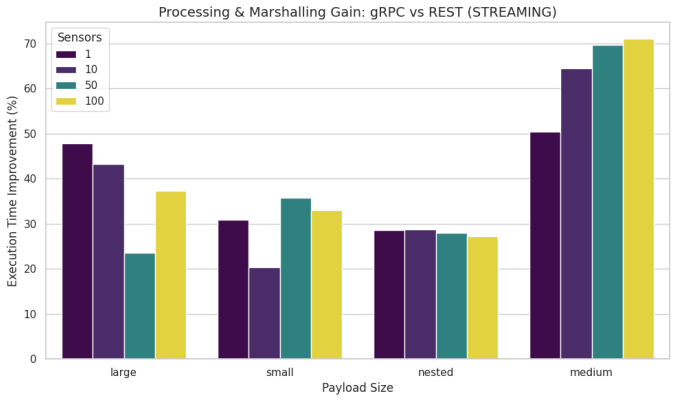


Fig. 7. Marshalling gain % (gRPC vs REST).

As highlighted in Fig. 7, the marshaling gain is exceptionally high for medium loads. In this scenario, gRPC demonstrates efficiency of over 70%, confirming that managing dense lists of binary strings is dramatically less expensive than generating JSON strings. Even in scenarios with highly complex data (nested) or large volumes of numeric data (large), gRPC maintains a consistent advantage of between 30% and 45%. This phenomenon explains the "paradox" observed in scalability tests: gRPC is able to process and transmit a Medium payload in less time than REST takes for a Small payload, since the savings in CPU cycles during serialization more than compensates for the greater volume of data transported.

5 Further evolutions

This study provides a solid foundation for comparing REST and gRPC under ideal network conditions. However, real-world telemetry systems usually operate on unstable networks, such as cellular (LTE/5G) or satellite connections. To further explore the analysis future benchmarks could be on this topics:

- **Network Impairment**: The current architecture is based on Docker that allows artificial alterations of the network through different tools. This could open to the observation of the system in case of:
  - **Packet Loss**: Analyzing the behavior of gRPC flow control and binary streams compared to REST when packets are lost;
  - **Throttling**: Evaluating performance in narrow-bandwidth scenarios (e.g. NB-IoT), where gRPC payload efficiency should provide an even more significant advantage;
  - **Latency and Jitter**: Simulating high response times (RTTs) to measure the impact of HTTP/2 multiplexing in geographically distributed systems.
- **System Resource Profiling**: This study focused on metrics correlated to the transport of data, a natural next step would be to systematically analyze CPU and RAM consumption at the container level. Using tools like Docker Stats, it would be possible to quantify the tradeoff between gRPC transmission efficiency and the computational cost required to run the HTTP/2 stack on limited hardware;
- **Security Overhead**:Future benchmarks may include the impact of TLS/mTLS encryption on both protocols. Since gRPC typically requires TLS in production environments, measuring the handshake and encryption overhead is essential for a comprehensive comparison that is ready for real-world deployment.

6 Conclusions

The study conducted with this project has clearly outlined the performance gap between the traditional REST architecture and the gRPC framework in the context of telemetry for distributed systems. Through a rigorous benchmark suite, it has been demonstrated that gRPC represents not just an incremental evolution but a true paradigm shift for high-frequency communications. Experimental results highlight three key pillars of the superiority of gRPC:

- **Data Efficiency**: The adoption of Protocol Buffers and HPACK compression enables a payload reduction that, in the most favorable use cases, exceeds 76% compared to the JSON format, ensuring critical bandwidth savings for large-scale infrastructures;
- **Scalability and Latency**: Thanks to HTTP/2 multiplexing, gRPC demonstrates exceptional stability as load increases, maintaining lower response times even with 100 simultaneous sensors;
- **CPU Optimization**: Latency breakdown analysis confirmed that the computational cost of marshaling is almost zero in gRPC, eliminating the text parsing bottleneck typical of REST.

Despite its clear performance superiority, gRPC adoption still presents challenges: its implementation in the IoT environment can be complex for devices with extremely limited resources that struggle to handle the HTTP/2 stack, while integration with web browsers remains not natively supported, often requiring additional components such as gRPC-Web or dedicated proxies (e.g. Envoy) to overcome current browser limitations.

In conclusion, the data collected in this study strongly support the adoption of gRPC for all applications where low latency and efficiency are stringent requirements. Although gRPC remains the preferred choice for high-performance telemetry systems and native cloud infrastructures—offering a robust, scalable architecture ready for the challenges of real-time systems—its adoption requires careful evaluation in resource-constrained scenarios. Although REST remains a

valid standard due to its ubiquity and extreme simplicity, the performance gap highlighted by the benchmarks positions gRPC as the preferred protocol for modern telemetry. However, it is crucial to recognize that gRPC's "lightweight" nature lies primarily in the data transport, not the stack required to manage it: the inherent complexity of HTTP/2 (stream management, HPACK compression, and state control) can be computationally expensive for minimal hardware or low-end microcontrollers. In these contexts, REST's implementation simplicity prevails over pure transport efficiency, making it still the most accessible option for extremely simple devices. Ultimately, gRPC emerges as the new benchmark for industrial monitoring and intensive data transfer, provided the hosting infrastructure is able to support the computational burden of such an advanced protocol.

References

[1] Google. 2024. What is gRPC? Retrieved April 22, 2026 from https://grpc.io
[2] Dom Harrington. 2024. *The History of REST APIs.* Retrieved April 22, 2026 from https://readme.com/resources/the-history-of-rest-apis
[3] Tullio Vardanega. 2025. Distributed communications. Retrieved April 22, 2026 from https://www.math.unipd.it/~tullio/SCD/2025/Material/T11.pdf

Received 24th April 2026